Local search algorithms

Chapter 4, Sections 3–4

Outline

- Hill-climbing
- Simulated annealing
- Genetic algorithms (briefly)
- Local search in continuous spaces (very briefly)

Local Search:

If:

- Uninformed search = searching in darkness
- Informed search = searching with a torch

Then:

Local search = climbing a hill while only seeing nearby ground

You do **not** see the whole landscape.

You only compare your current position with its neighbors and move to a better one.

You are not building a tree.
You are just **improving the current state step by step**.

How is Local Search Different from Uninformed & Informed Search?

Feature	Uninformed Search	Informed Search	Local Search
Builds search tree?	Yes	Yes	✗ No
Keeps paths?	Yes	Yes	✗ No
Uses heuristic?	No	Yes	Usually yes (objective function)
Memory usage	Large	Large	Very small
Goal	Reach goal state	Reach goal efficiently	Improve state (optimization)
Completeness	Sometimes	Sometimes	Usually not
Optimality	Sometimes	Sometimes	Not guaranteed

Iterative improvement algorithms

In many optimization problems, path is irrelevant;
the goal state itself is the solution

Then state space = set of "complete" configurations
(complete-state formulation vs. incremental formulation)

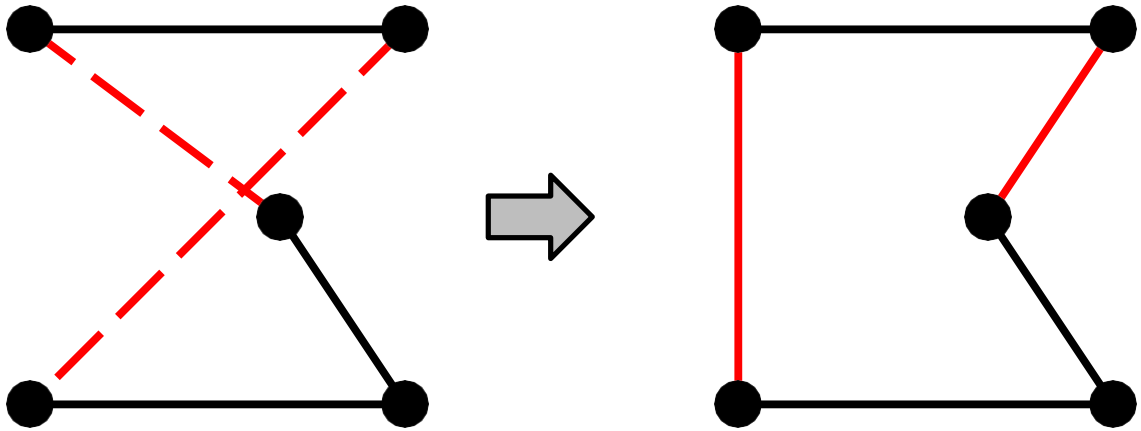
In such cases, can use iterative improvement algorithms;
keep a single "current" state, try to improve it

Constant space, suitable for online as well as offline search

Often want to find optimal configuration, e.g., TSP,
but also works for constraint satisfaction problems, e.g. *n*queens, timetabling

Example: Travelling Salesperson Problem

Start with any complete tour, perform pairwise exchanges

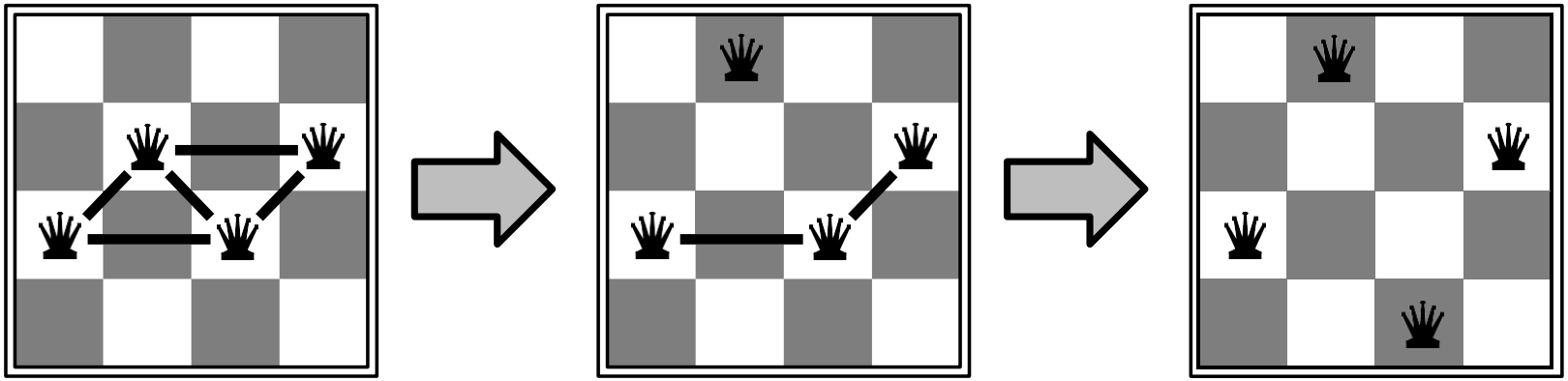


Variants of this approach get within 1% of optimal very quickly with thousands of cities

Example: n-queens

Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal

Local search: start with all n, move a queen to reduce conflicts



h = 5 **h = 2** **h = 0**

Almost always solves n-queens problems almost instantaneously for very large n, e.g., $n = 1$ *million*

(Why? Perhaps because choices for queen k are made wrt all others)

What is Hill Climbing?

Hill Climbing is a **local search algorithm** used in Artificial Intelligence for **optimization problems**. The goal is to reach the **highest point (maximum)** in a search space (or lowest point for minimization). Think of it as climbing a hill: at each step, you move **upwards towards a higher value**, until you can't go higher.

- It's **greedy**: always chooses the best neighbor.
- It doesn't maintain a global view; it only knows the **current state** and its neighbors.

Algorithm (Simple Form)

1. Start from a **random initial state**.
2. Evaluate the **value of the current state** (using a heuristic function h(n)).
3. Find **neighboring states**.
4. Move to the neighbor with the **highest value**.
5. Repeat until **no neighbor has a higher value** (local maximum is reached).

Hill-climbing (or gradient ascent/descent)

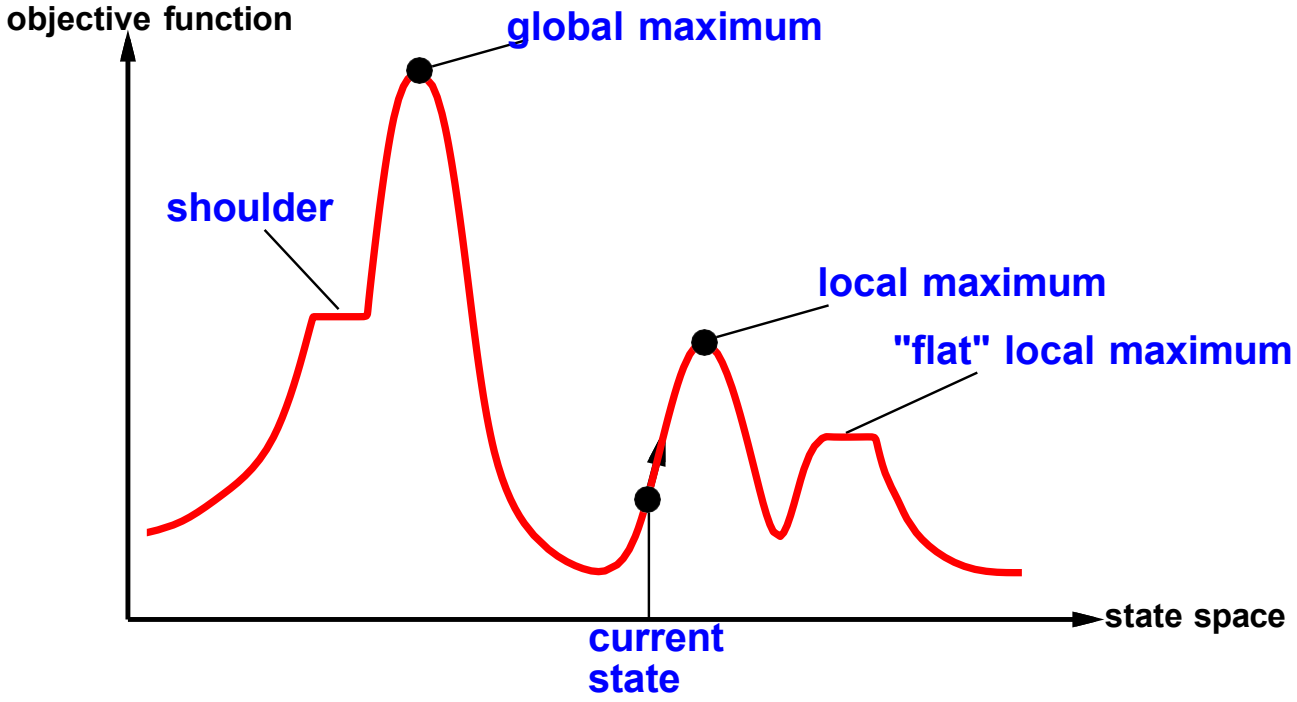
"Like climbing Everest in thick fog with amnesia"

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    inputs: problem, a problem
    local variables: current, a node
                     neighbor, a node

    current ← MAKE-NODE(INITIAL-STATE[problem])
    loop do
        neighbor ← a highest-valued successor of current
        if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
        current ← neighbor
    end
```

Hill-climbing contd.

Useful to consider state space landscape



Random-restart hill climbing overcomes local maxima—trivially complete

Random sideways moves ☺ escape from shoulders ☹ loop on flat maxima

Problems with Hill Climbing

Hill Climbing is simple and fast, but it faces **major challenges** in complex search spaces.

(a) Local Maxima

- Occurs when a **neighboring state is worse**, but the current state is **not the global maximum**.
- Analogy: climbing a **small hill**, thinking you've reached the top, while a **taller hill exists elsewhere**.

Example:

h values along a path: $1 \rightarrow 4 \rightarrow 6 \rightarrow 5 \rightarrow 7$

Current state: 6 (neighbors: 5 and 6)

Hill climbing stops at $6 \rightarrow$ Local maximum

(b) Plateaus

- A **flat area in the search space** where all neighboring states have the **same value**.
- The algorithm can **wander randomly** or **stall** because no better neighbor exists.

Example:

h values: 3 → 3 → 3 → 4

Current state: 3

Neighbors: all 3 → plateau

(c) Ridges

- Narrow regions where the **global maximum lies diagonally**.
- Hill climbing moves **one dimension at a time**, so it may **oscillate** and struggle to ascend.

Example (2D space):

- Global maximum lies at (x=5, y=5).
- Current position (x=1, y=1).
- Moving only along x or y makes small progress, but zigzagging is needed.

Shoulder

- A **shoulder** is a **flat region (plateau)** in the search space where:
  - The current state has **no better neighbors** (all neighbors have equal heuristic value).
  - But unlike a plateau that leads to a dead end, the **flat area eventually leads upward** toward the global maximum.

Think of it as the **side of a hill**:

- You move across a flat region (shoulder), and then the slope rises again toward the peak.

Hill Climbing Example

Problem: Maximize the function

$f(x)=-(x-3)2+9$

- Search space: x=0,1,2,3,4,5, Heuristic: f(x) (higher is better)

Step 1: Table of Heuristic Values

x (Current State)	f(x) (Heuristic)	Neighbors	Next Move
0	0	1	1 (f=5)
1	5	0,2	2 (f=8)
2	8	1,3	3 (f=9)
3	9	2,4	Stop → local maximum
4	8	3,5	3 (already higher)
5	5	4,6	4 (f=8)
6	0	5	5 (f=5)

Activity

1. **Pick a random start:** x = 0, 1, or 5
2. **Find neighbors**: x-1, x+1
3. **Move to the neighbor with higher f(x)**
4. **Repeat** until no neighbor is higher → local maximum
5. **Answer questions:**
    - Did you reach global maximum?
    - What happens if you start at x=6?

Simulated annealing

Idea: escape local maxima by allowing some "bad" moves
Simulated annealing is a **local search algorithm** designed to overcome the main weakness of hill-climbing:

Getting stuck in local maxima.
It is a probabilistic version of hill-climbing that sometimes allows **moves to worse states**.

but gradually decrease their size and frequency

Annealing?

The name comes from metallurgy:

- Annealing is a process where a material is heated to high temperature.
- Then it is cooled down slowly.
- Slow cooling allows atoms to settle into a **low-energy stable configuration**.

Similarly, simulated annealing:

- Starts with a "high temperature" (more randomness).
- Gradually lowers the temperature.
- Eventually behaves like hill-climbing.

How It Works (According to the Book)

At each step:

1. Pick a random successor.
2. If successor is better ⟶ move to it.
3. If successor is worse ⟶ move to it with some probability.

The probability of accepting a worse move depends on:

- How much worse it is.
- The current temperature T.

A worse move is accepted with probability:

$$e^{\Delta E/T}$$

Where:

- ΔE = change in value (negative if worse)
- T = temperature

As T decreases:

- The probability of accepting worse moves decreases.

When T ⟶ 0:

- The algorithm becomes identical to hill-climbing.

```
function SIMULATED-ANNEALING (problem, schedule) returns a solution state
    inputs: problem, a problem
            schedule, a mapping from time to "temperature"
    local variables: current, a node
                     next, a node
                     T, a "temperature" controlling prob. of downward steps

    current ← MAKE-NODE(INITIAL-STATE[problem])
    for t ← 1 to ∞ do
        T ← schedule[t]
        if T = 0 then return current
        next ← a randomly selected successor of current
        ΔE ← VALUE[next] − VALUE[current]
        if ΔE > 0 then current ← next
        else current ← next only with probability e^(Δ E/T)
```

Role of Temperature

Temperature controls randomness.

High Temperature:

- Large probability of accepting worse states.
- Search behaves randomly.
- Escapes local maxima easily.

Low Temperature:

- Very unlikely to accept worse states.
- Behaves like greedy hill-climbing.
- Converges toward a solution.

Properties of simulated annealing

At fixed "temperature" T, state occupation probability reaches Boltzman distribution

$$p(x) = \alpha e^{\frac{E(x)}{kT}}$$

The probability of the system being in a particular state at a fixed temperature.

It is given by:

Where:

E(x) = value (or energy) of state x

T = temperature

k = Boltzmann constant

α\alpha= normalization constant (so probabilities sum to 1)

At a fixed temperature:

States with higher E(x)) have higher probability.

States with lower E(x) still have some probability.

The distribution depends strongly on temperature.

Role of Temperature in the Distribution

High Temperature (T large)

- Differences in E(x) matter less.
- All states have similar probability.
- Distribution is “flat”.

  Search behaves almost randomly.

Low Temperature (T small)

- Differences in E(x) are magnified.
- Best states dominate probability.
- Distribution becomes sharply peaked.

  As T→0:
- Probability concentrates on the optimal state.

Why Is This Important in Simulated Annealing?

If we hold temperature fixed and run long enough,
the probability of being in each state approaches the Boltzmann distribution.
As we slowly decrease temperature:

- The distribution increasingly favors better states.
- Eventually, the optimal state dominates.

Numerical Example

Suppose:

Current state value = 20
New state value = 15

ΔE=−5\Delta E = -5 ΔE=−5

Case 1: High Temperature (T = 50)

e−5/50=e−0.1≈0.90

90% chance of accepting worse state.

Exploration dominates.

Case 2: Low Temperature (T = 1)

e−5/1=e−5≈0.0067

Less than 1% chance.

Greedy behavior dominates.

T decreased slowly enough $\Rightarrow$ always reach best state x^*

because $e^{\frac{E(x^*)}{kT}} / e^{\frac{E(x)}{kT}} = e^{\frac{E(x^*)-E(x)}{kT}}$ 1 for small T

Is this necessarily an interesting guarantee??

Devised by Metropolis et al., 1953, for physical process modelling

Widely used in VLSI layout, airline scheduling, etc.

Local beam search

Idea: keep k states instead of 1; choose top k of all their successors

Not the same as k searches run in parallel!
Searches that find good states recruit other searches to join them

Problem: quite often, all k states end up on same local hill

Idea: choose k successors randomly, biased towards good ones

Observe the close analogy to natural selection!

Example

Let's maximize a function f(x).

States and values:

State	Value
A	4
B	7
C	5
D	10
E	6
F	12
G	9

Assume:

k = 2 (beam width = 2)

Step 1: Initial States

Randomly choose:

A (4)

C (5)

Current beam:

{ A, C }

Step 2: Generate Successors

Successors of A → B(7), E(6)

Successors of C → F(12), G(9)

All successors:

B(7), E(6), F(12), G(9)

Select best k = 2:

F(12), G(9)

New beam:

{ F, G }

Step 3: Next Iteration

Suppose successors:

F $\rightarrow$ D(10)

G $\rightarrow$ B(7)

All successors:

D(10), B(7)

Select best 2:

D(10), B(7)

New beam:

{ D, B }

Continue until stopping condition.

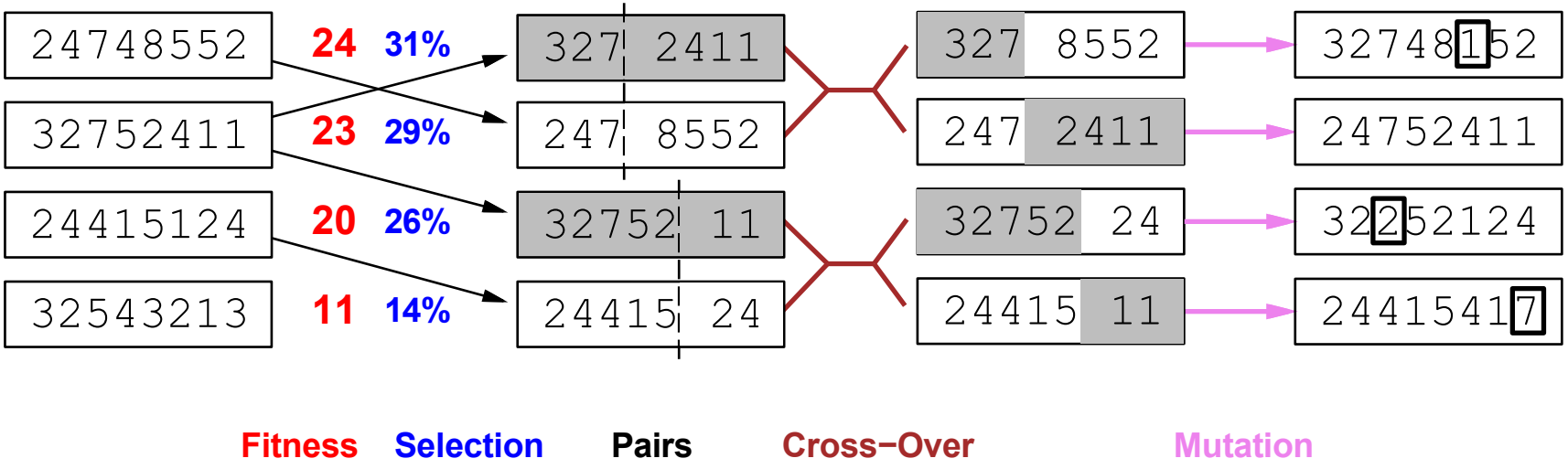
- Initially weak states (A and C) were discarded quickly.
- Strong state F (12) dominated search.
- The algorithm moved entire beam toward high-value region.

This shows:

Beam search concentrates resources on promising areas.

Genetic algorithms

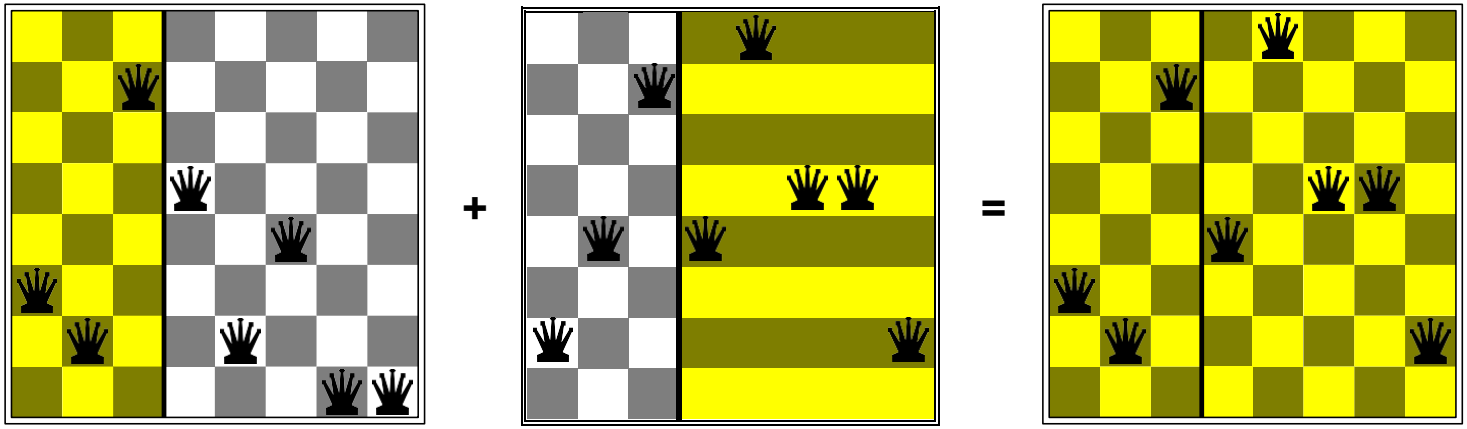
= stochastic local beam search + generate successors from pairs of states



Genetic algorithms contd.

GAs require states encoded as strings (GPs use programs)

Crossover helps iff substrings are meaningful components

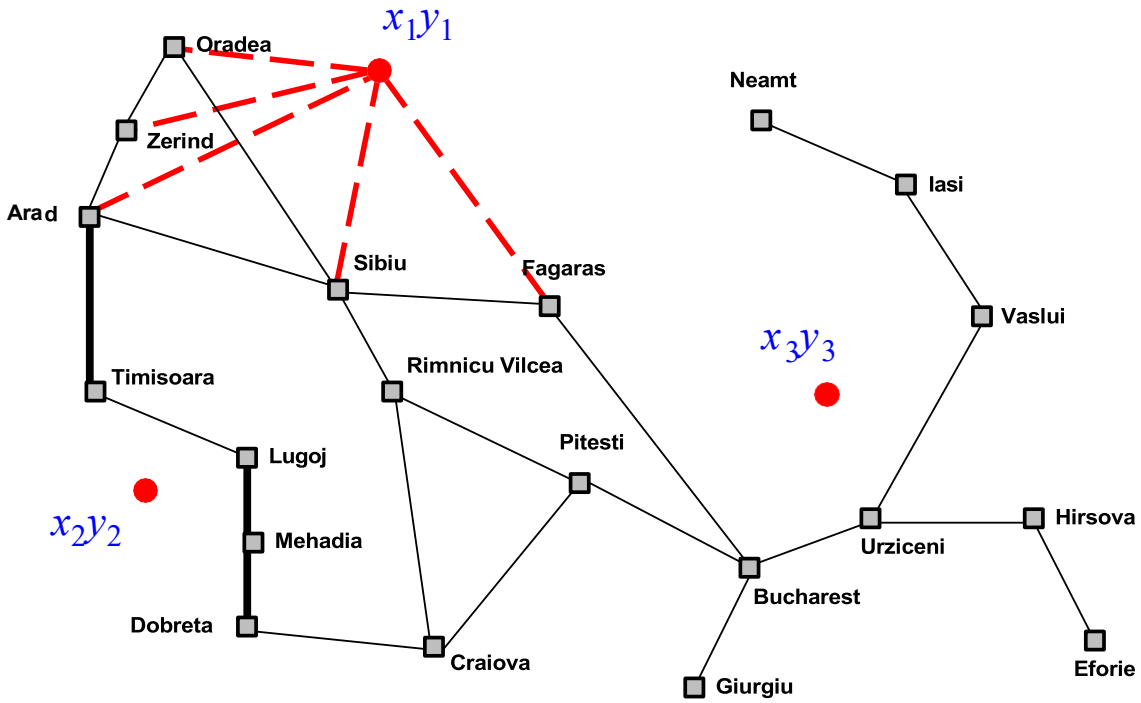


GAs ≠ evolution: e.g., real genes encode replication machinery!

Continuous state spaces

Suppose we want to site three airports in Romania:

- 6-D state space defined by (x_1, y_2), (x_2, y_2), (x_3, y_3)
- objective function $f(x_1, y_2, x_2, y_2, x_3, y_3)$ =
  sum of squared distances from each city to nearest airport



Search methods

Discretization methods turn continuous space into discrete space,
e.g., empirical gradient considers $\pm\delta$ change in each coordinate

Gradient methods compute

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3}\right]$$

to increase/reduce f, e.g., $\mathrm{x} \leftarrow \mathrm{x} + \alpha \nabla f(\mathrm{x})$

Locally, $f(x_1, y_2, x_2, y_2, x_3, y_3) = (x_1 - x_{\text{Arad}})^2 + (y_1 - y_{\text{Arad}})^2 + \cdots$, so

$$\frac{\partial f}{\partial x_1} = 2(x_1 - x_{\text{Arad}}) + 2(x_1 - x_{\text{Sibiu}}) + \cdots$$

Sometimes can solve for $\nabla f(\mathrm{x}) = 0$ exactly (e.g., with one airport).
Newton–Raphson (1664, 1690) iterates $\mathrm{x} \leftarrow \mathrm{x} - \mathrm{H}^{-1}(\mathrm{x}) \nabla f(\mathrm{x})$
to solve $\nabla f(\mathrm{x}) = 0$, where $\mathrm{H}_{ij} = \partial^2 f / \partial x_i \partial x_j$